Project 2: Key-Value Store

Cloud Computing (Spring 2026)

Student Name: Adewole Adeoshun
CUNY ID: 24081306
Student Type: Undergraduate Student
Professor: Jun Li

Tasks Completed: 1-6 (including Graduate Extension)
Due Date: April 9, 2026

Cloud Provider: AWS (us-east-1)
Redis Version: 7.x
EC2 Instance Type: t3.micro (Amazon Linux 2023)

TASK 1: CLOUD ENVIRONMENT SETUP

Cloud Provider & Instance Configuration:

AWS (Amazon Web Services) was selected using the us-east-1 (N. Virginia) region. Six EC2 instances of type t3.micro were provisioned within the same Availability Zone (us-east-1f) to minimize inter-node latency. Each instance ran Amazon Linux 2023.

Instance ID	Private IP	Initial Role	Type	AZ
i-0c683f1b878613e36	172.31.70.178	Master 1 (slots 0-5460)	t3.micro	us-east-1f
i-01080f5c7460fdbd5	172.31.79.150	Master 2 (slots 5461-10922)	t3.micro	us-east-1f
i-0f067f07f63ccbaee	172.31.75.162	Master 3 (slots 10923-16383)	t3.micro	us-east-1f
i-0dd67b8fb6e050fcb	172.31.69.248	Replica of Master 1	t3.micro	us-east-1f
i-0e1b2c6858479b8e7	172.31.68.47	Replica of Master 2	t3.micro	us-east-1f
i-03ba6c22d4a9d0bfe	172.31.69.93	Replica of Master 3	t3.micro	us-east-1f

Commands Run:

Start all 6 instances:

```
aws ec2 start-instances --region us-east-1 --instance-ids \
  i-0c683f1b878613e36 i-01080f5c7460fdbd5 i-0f067f07f63ccbaee \
  i-0dd67b8fb6e050fcb i-0e1b2c6858479b8e7 i-03ba6c22d4a9d0bfe
```

Verify instances running:

```
aws ec2 describe-instances --region us-east-1 \
  --query 'Reservations[*].Instances[*].[InstanceId,State.Name,PublicIpAddress]' \
  --output table
```

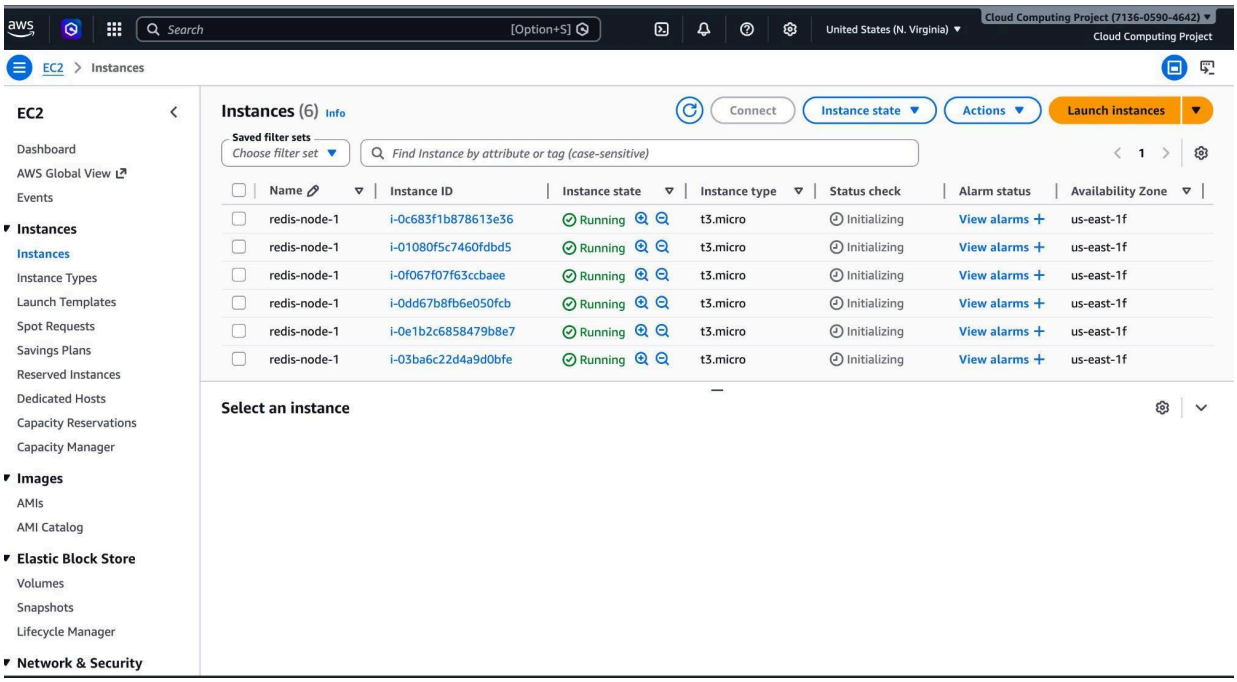
Output:

```
-----------------------------------------------------
|               DescribeInstances                   |
+----------------------+----------+-----------------+
|  i-0c683f1b878613e36 |  running |  98.93.75.187   |
|  i-01080f5c7460fdbd5 |  running |  98.82.131.63   |
|  i-0f067f07f63ccbaee |  running |  3.238.233.92   |
|  i-0dd67b8fb6e050fcb |  running |  34.204.186.109 |
|  i-0e1b2c6858479b8e7 |  running |  3.239.112.212  |
|  i-03ba6c22d4a9d0bfe |  running |  100.56.16.86   |
+----------------------+----------+-----------------+
```

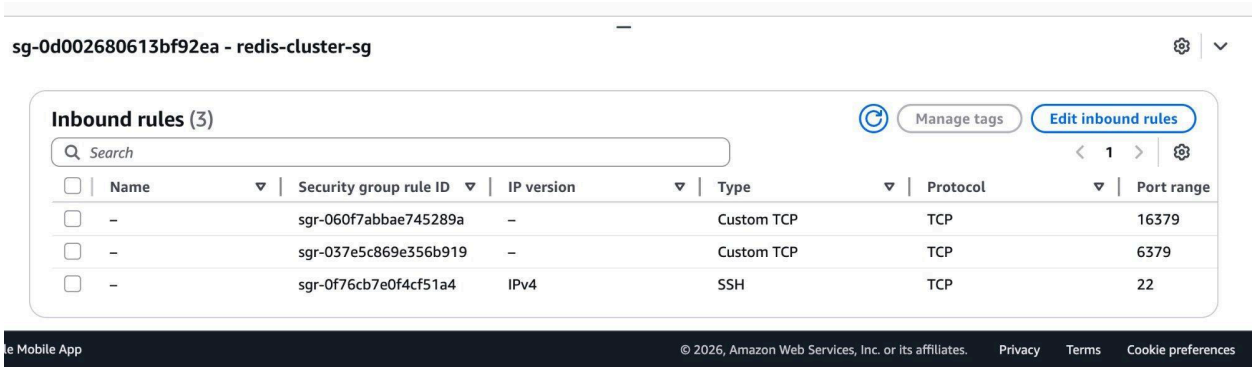
Security Group Configuration (redis-cluster-sg):

Port	Purpose
6379	Redis client connections — all SET/GET/DEL operations
16379	Redis cluster bus — gossip, failure detection, config propagation (Redis port + 10000)
22	SSH access for node administration

Screenshots for Task 1:



All 6 EC2 instances in Running state in AWS Console (us-east-1f)



Security group redis-cluster-sg (inbound rules for ports 6379, 16379, and 22)

Explanation:

I chose AWS because of its familiar CLI tooling and its free-tier eligibility on t3.micro instances. Six nodes were deployed to meet the 3-6 VM requirement with a full 3-master + 3-replica topology. Placing all instances in us-east-1f minimizes network latency between nodes, which directly speeds up cluster gossip and failover detection.

The security group required exactly three rules.

Port 6379 is the Redis client port where all key-value operations are handled.

Port 16379 is the cluster bus port , Redis always uses client port + 10,000 for inter-node heartbeat messages, failure detection votes, and hash slot migrations. Without port 16379, nodes cannot form a cluster.

Port 22 allows SSH for configuration and troubleshooting.

TASK 2: BASIC REDIS CLUSTER DEPLOYMENT

Commands Run:

Step 1: Install Redis (all 6 nodes)

```
sudo yum install redis -y
sudo redis-server /etc/redis/redis.conf
```

Redis Configuration File (/etc/redis/redis.conf):

```
bind 0.0.0.0              # accept connections from all interfaces (required for
inter-node)
port 6379
cluster-enabled yes       # enable cluster mode
cluster-config-file nodes.conf
cluster-node-timeout 5000  # 5s before failed master triggers replica election
appendonly yes            # enable AOF persistence
save 3600 1               # RDB: save if 1+ keys changed in 3600s
save 300 100              # RDB: save if 100+ keys changed in 300s
save 60 10000             # RDB: save if 10000+ keys changed in 60s
```

Step 2: Initialize Redis Cluster

```
redis-cli --cluster create \
  172.31.70.178:6379 172.31.79.150:6379 172.31.75.162:6379 \
  172.31.69.248:6379 172.31.68.47:6379  172.31.69.93:6379 \
  --cluster-replicas 1
```

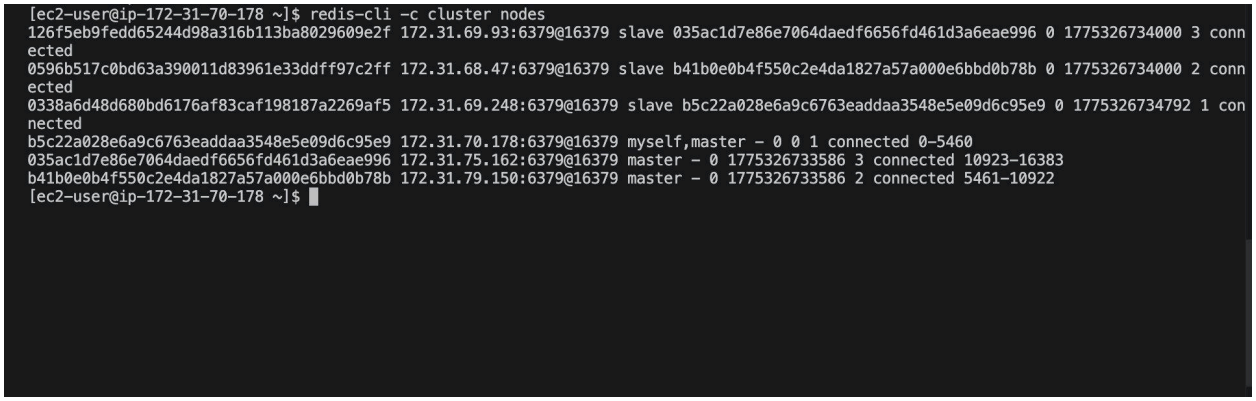
Step 3: Verify cluster status

```
redis-cli -c cluster info
redis-cli -c cluster nodes
```

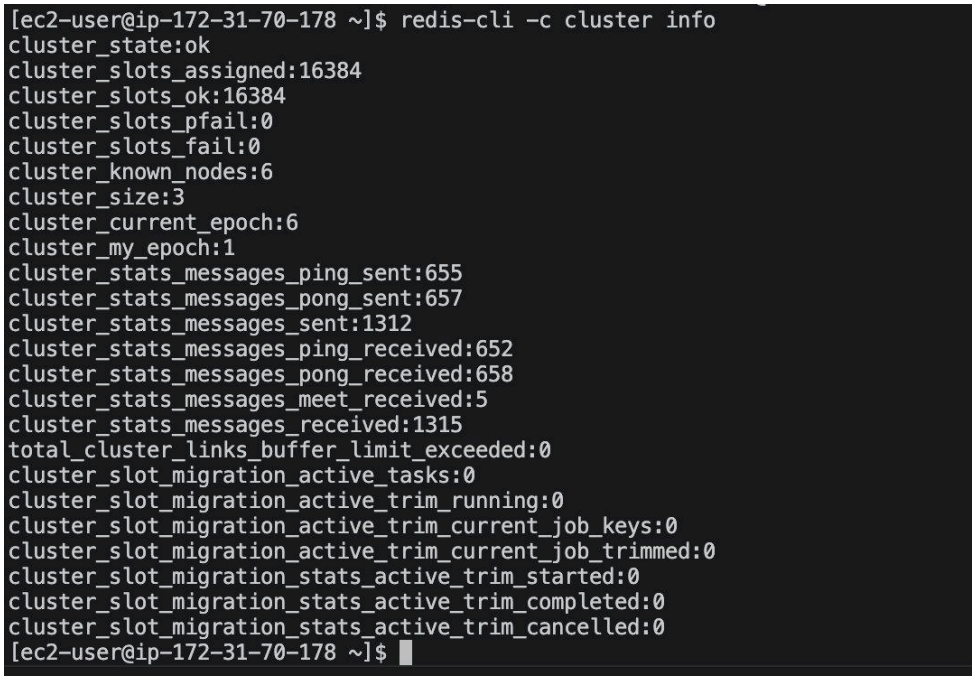
Output (cluster info):

```
cluster_state:ok
cluster_slots_assigned:16384
cluster_slots_ok:16384
cluster_known_nodes:6
cluster_size:3
```

Screenshots for Task 2:



cluster nodes + 3 masters with hash slot ranges assigned, 3 replicas all connected



cluster info — cluster_state:ok, all 16,384 slots assigned, cluster_size:3

Explanation:

I chose a 6-node setup (3 masters + 3 replicas) because it satisfies the minimum Redis Cluster requirement of 3 masters while giving each master a standby replica for automatic failover. The --cluster-replicas 1 flag pairs exactly one replica per master.

Setting bind 0.0.0.0 is critical because it allows Redis to accept connections from any IP, enabling inter-node communication across private IPs. Binding to localhost would prevent cluster formation. The cluster-node-timeout 5000 sets the failure detection window to 5 seconds, enabling fast failover without triggering unnecessary elections.

Redis automatically distributed all 16,384 hash slots: Node 1 (0-5460), Node 2 (5461-10922), Node 3 (10923-16383). cluster_state:ok with cluster_slots_ok:16384 confirms full operation.

TASK 3: FAILOVER CONFIGURATION AND TESTING

Automatic Failover Confirmation:

Redis Cluster automatic failover is enabled by default when cluster-enabled yes is configured. No additional configuration was required. The cluster-node-timeout 5000 controls how long a master must be unreachable before replicas initiate a leader election.

Commands Run:

Step 1: Record cluster state BEFORE failover

```
redis-cli -c cluster nodes
```

Step 2: Kill Master 2 (172.31.79.150 — slots 5461-10922)

```
sudo pkill redis-server     # executed on node 172.31.79.150
```

Step 3: Watch failover from Node 1

```
redis-cli -c cluster nodes  # run repeatedly until promotion complete
```

Step 4: Verify cluster still accepts key operations

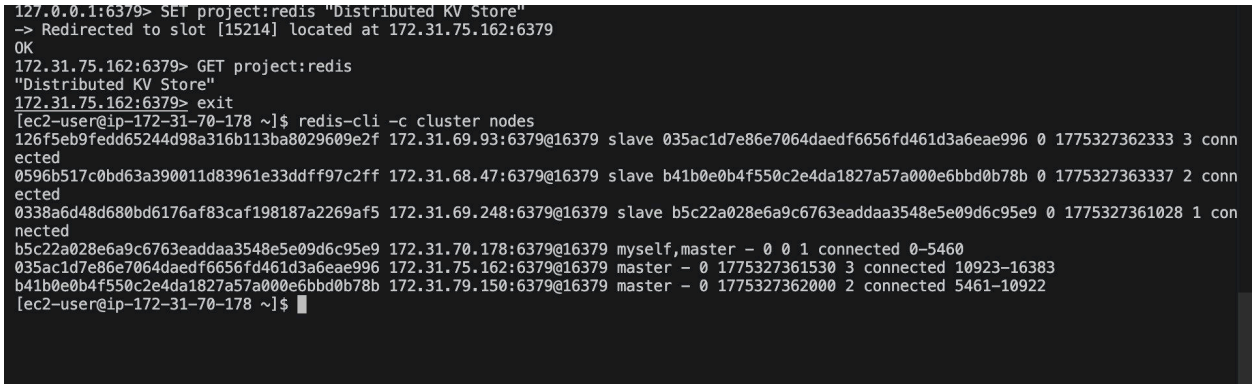
```
redis-cli -c
SET failover:test "Cluster survived!"
GET failover:test
exit
```

Step 5: Restart Node 2 and verify it rejoins as replica

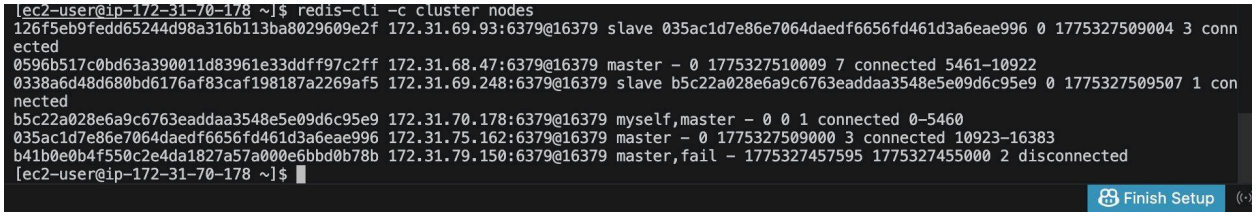
```
sudo redis-server /etc/redis/redis.conf
redis-cli -c cluster nodes
```

Event	Result
Master 2 killed (172.31.79.150)	Node shows master,fail — slots 5461-10922 briefly unavailable
Replica promoted (172.31.68.47)	Automatically promoted to master — full slot coverage restored
Node 2 restarted	Rejoined as slave under new master — cluster back to 3+3 topology

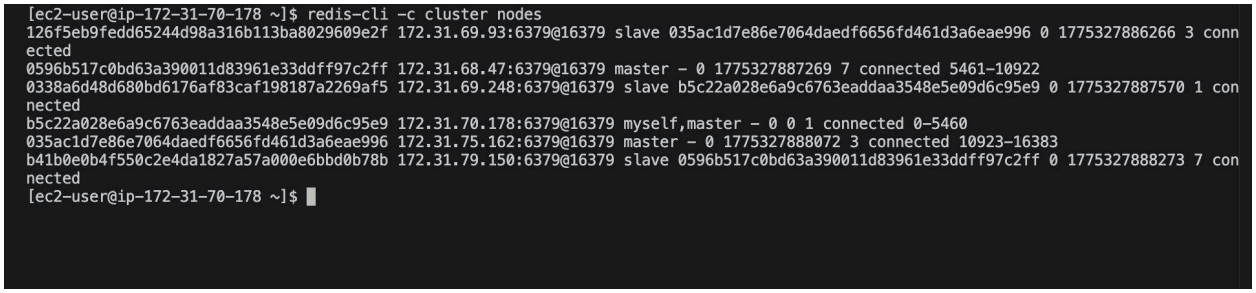
Screenshots for Task 3:



BEFORE — Node 2 (172.31.79.150) is master owning slots 5461-10922; Node 6 is its replica



AFTER — Node 2 shows master,fail; Node 6 (172.31.68.47) automatically promoted to master



RECOVERY — Node 2 rejoined as slave under Node 6; full 3+3 topology restored

Explanation:

When Master 2 was killed, the remaining nodes detected its absence within the cluster-node-timeout window (5 seconds). Node 6 — the replica of Master 2 — initiated a leader election, received votes from the other two masters, and was automatically promoted to master, inheriting responsibility for hash slots 5461-10922.

Client operations continued on all other slots during the failover. Operations targeting slots 5461-10922 experienced a brief pause (~5 seconds) during the election but resumed once Node 6 was promoted. After restarting Node 2, it automatically rejoined as a replica under Node 6, restoring the full fault-tolerant topology without manual reconfiguration.

TASK 4: KEY-VALUE WORKLOAD GENERATOR

Workload Generator Source Code (workload_generator.py):

```
import redis, time, random, string, threading, statistics, json
from datetime import datetime

latencies = []; errors = 0; lock = threading.Lock()

def random_string(n=8):
    return ''.join(random.choices(string.ascii_lowercase, k=n))

def run_workload(client, ops_per_thread, thread_id):
    global errors
    for i in range(ops_per_thread):
        key = f'key:{thread_id}:{i}'
        val = random_string()
        op = random.choice(['set', 'get', 'del'])
        try:
            start = time.time()
            if op == 'set':   client.set(key, val)
```

```
            elif op == 'get': client.get(key)
            elif op == 'del': client.delete(key)
            latency = (time.time() - start) * 1000
            with lock: latencies.append(latency)
        except Exception:
            with lock: errors += 1

def main(total_ops=10000, num_threads=10):
    client = redis.RedisCluster(
        host='172.31.70.178', port=6379, decode_responses=True)
    ops_per_thread = total_ops // num_threads
    threads = [threading.Thread(
        target=run_workload, args=(client, ops_per_thread, i))
               for i in range(num_threads)]
    start_time = time.time()
    for t in threads: t.start()
    for t in threads: t.join()
    duration = time.time() - start_time
    throughput = len(latencies) / duration
    # print metrics + save to baseline_results.json

if __name__ == '__main__': main()
```

Failover Test Generator Source Code (failover_test.py):

```
import redis, time, random, threading, json

latencies = []; errors = 0; lock = threading.Lock(); running = True

def run_workload(client, thread_id):
    global errors, running
    i = 0
    while running:
        key = f'key:{thread_id}:{i}'
        op = random.choice(['set', 'get', 'del'])
        try:
            start = time.time()
            if op == 'set':   client.set(key, 'val')
            elif op == 'get': client.get(key)
            elif op == 'del': client.delete(key)
            with lock: latencies.append((time.time() - start) * 1000)
        except:
            with lock: errors += 1
        i += 1

def main():
    global running
    client = redis.RedisCluster(
        host='172.31.70.178', port=6379, decode_responses=True,
        retry_on_error=[redis.exceptions.ConnectionError],
        retry=redis.retry.Retry(redis.backoff.ExponentialBackoff(), 6))
    threads = [threading.Thread(target=run_workload, args=(client, i))
               for i in range(10)]
    for t in threads: t.start()
    print('Workload running... KILL A MASTER NODE NOW!')
    time.sleep(30); running = False
    for t in threads: t.join()
    # print metrics + save to failover_results.json

if __name__ == '__main__': main()
```

Explanation:

I used redis.RedisCluster because it is cluster-aware, it automatically routes each key operation to the correct master node based on the CRC16 hash of the key modulo 16,384. A plain redis.Redis() client would fail with MOVED errors when keys hash to slots on other masters.

The generator uses 10 concurrent threads to simulate multiple simultaneous clients, distributing operations across all three shards. Each thread performs a random mix of SET, GET, and DEL on unique keys to ensure even hash slot distribution. Latency is measured per operation in milliseconds and throughput is total successful operations divided by elapsed time.

The failover generator adds retry logic with exponential backoff so the client automatically reconnects after master promotion completes, capturing the full latency spike rather than crashing.

TASK 5: PERFORMANCE EVALUATION

Baseline Performance Test:

The workload generator was run against the fully healthy 6-node cluster. 10,000 operations across 10 concurrent threads.

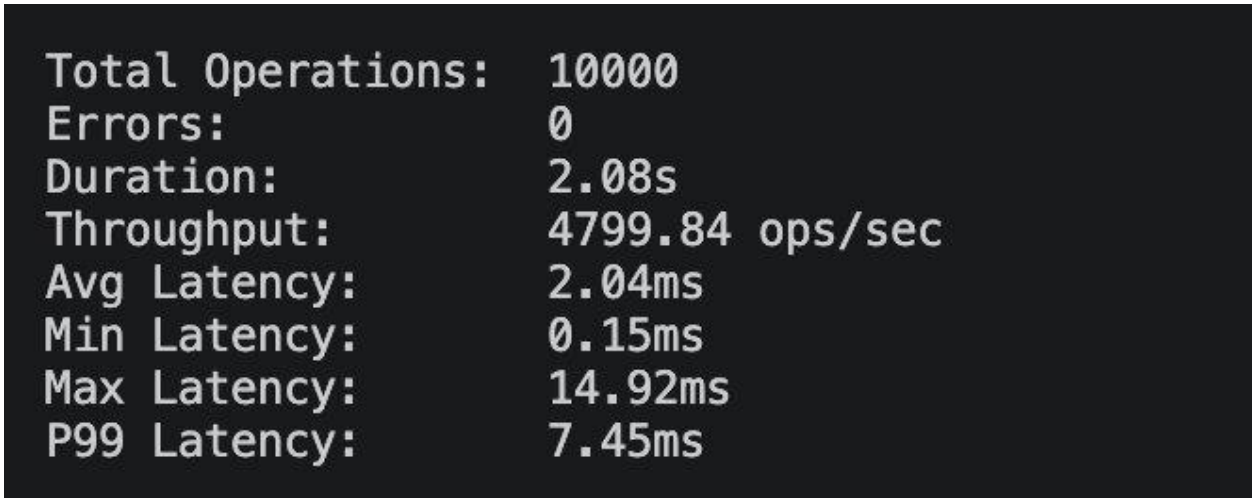
Command:

```
python3 ~/workload_generator.py
```

Output:

```
Total Operations:  10000
Errors:            0
Duration:          2.08s
Throughput:        4799.84 ops/sec
Avg Latency:       2.04ms
Min Latency:       0.15ms
Max Latency:       14.92ms
P99 Latency:       7.45ms
Results saved to baseline_results.json
```

Screenshot — Baseline Results:



Baseline test results — 10,000 ops, 0 errors, 4,799 ops/sec, 2.04ms avg latency

Failover Performance Test:

The failover workload ran for 30 seconds. ~10 seconds in, Master Node 2 (172.31.79.150) was killed on a second terminal.

Terminal 1 - start continuous workload:

```
python3 ~/failover_test.py
```

Terminal 2 - kill master mid-run:

```
sudo pkill redis-server    # on Node 2 (172.31.79.150)
```

Output:

```
Total Operations:  104796
Errors:            318
Duration:          30.05s
Throughput:        3487.76 ops/sec
Avg Latency:       2.21ms
Min Latency:       0.14ms
Max Latency:       1310.05ms
P99 Latency:       7.63ms
Results saved to failover_results.json
```

Screenshot — Failover Results:

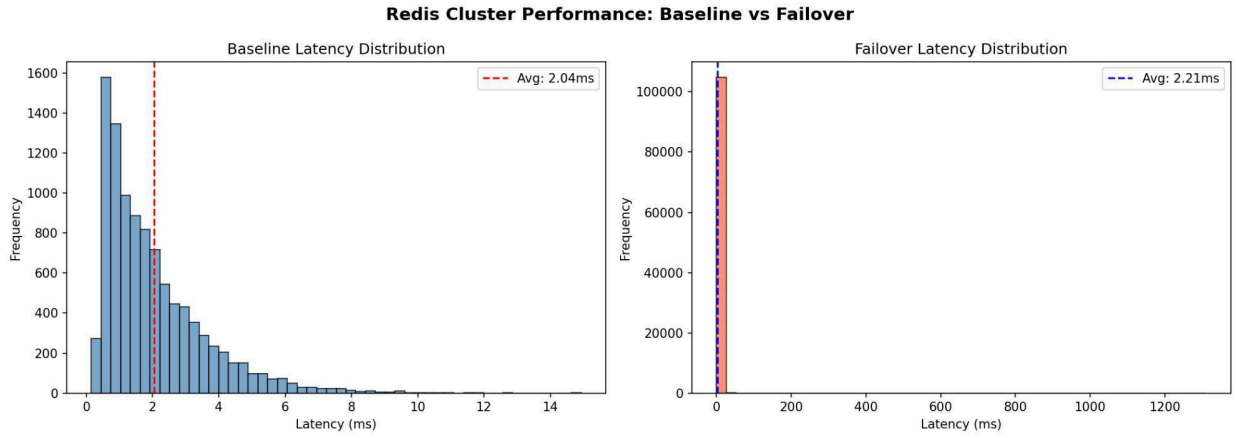
```
Workload running... KILL A MASTER NODE NOW (within 10 seconds)!
Run on another terminal: sudo pkill redis-server

Total Operations:  104796
Errors:            318
Duration:          30.05s
Throughput:        3487.76 ops/sec
Avg Latency:       2.21ms
Min Latency:       0.14ms
Max Latency:       1310.05ms
P99 Latency:       7.63ms
```

Failover results — 318 errors during failover window, 1,310ms max latency spike

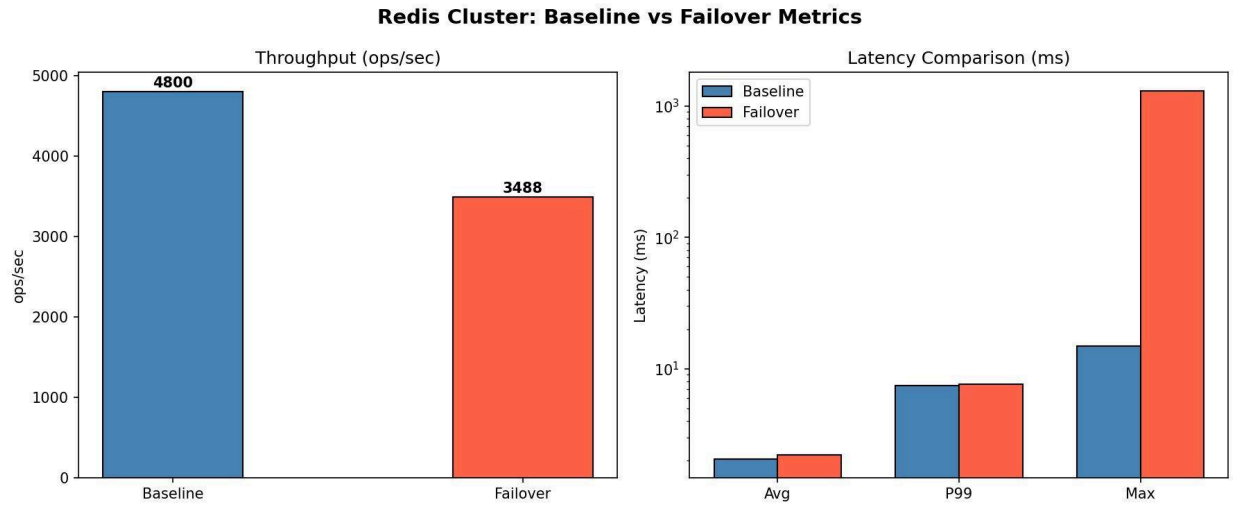
Performance Comparison:

Metric	Baseline	Failover	Delta
Throughput	4,799 ops/sec	3,487 ops/sec	-27.3%
Errors	0	318	0.3% error rate
Avg Latency	2.04ms	2.21ms	+8.3%
Max Latency	14.92ms	1,310ms	+87x spike
P99 Latency	7.45ms	7.63ms	+2.4%

Performance Graphs:



Latency distribution — baseline (tight 0-15ms) vs failover (1,310ms spike during election)



Throughput and latency comparison — log scale shows 87x max spike; avg/P99 nearly identical

Cluster State During Failover Test:

```
[ec2-user@ip-172-31-70-178 ~]$ redis-cli -c cluster nodes
126f5eb9fedd65244d98a316b113ba8029609e2f 172.31.69.93:6379@16379 slave 035ac1d7e86e7064daedf6656fd461d3a6eae996 0 1775624554000 3 conn
ected
035ac1d7e86e7064daedf6656fd461d3a6eae996 172.31.75.162:6379@16379 master - 0 1775624553000 3 connected 10923-16383
b5c22a028e6a9c6763eaddaa3548e5e09d6c95e9 172.31.70.178:6379@16379 myself,master - 0 0 1 connected 0-5460
b41b0e0b4f550c2e4da1827a57a000e6bbd0b78b 172.31.79.150:6379@16379 master,fail - 1775624382305 1775624380000 8 disconnected
0596b517c0bd63a390011d83961e33ddff97c2ff 172.31.68.47:6379@16379 master - 0 1775624553000 9 connected 5461-10922
0338a6d48d680bd6176af83caf198187a2269af5 172.31.69.248:6379@16379 slave b5c22a028e6a9c6763eaddaa3548e5e09d6c95e9 0 1775624554330 1 con
nected
[ec2-user@ip-172-31-70-178 ~]$
```

Cluster during failover test — Node 2 master,fail; Node 6 promoted to master owning slots 5461-10922

```
[ec2-user@ip-172-31-70-178 ~]$ redis-cli -c cluster nodes
126f5eb9fedd65244d98a316b113ba8029609e2f 172.31.69.93:6379@16379 slave 035ac1d7e86e7064daedf6656fd461d3a6eae996 0 1775624641000 3 conn
ected
035ac1d7e86e7064daedf6656fd461d3a6eae996 172.31.75.162:6379@16379 master - 0 1775624641518 3 connected 10923-16383
b5c22a028e6a9c6763eaddaa3548e5e09d6c95e9 172.31.70.178:6379@16379 myself,master - 0 0 1 connected 0-5460
b41b0e0b4f550c2e4da1827a57a000e6bbd0b78b 172.31.79.150:6379@16379 slave 0596b517c0bd63a390011d83961e33ddff97c2ff 0 1775624641000 9 con
nected
0596b517c0bd63a390011d83961e33ddff97c2ff 172.31.68.47:6379@16379 master - 0 1775624640816 9 connected 5461-10922
0338a6d48d680bd6176af83caf198187a2269af5 172.31.69.248:6379@16379 slave b5c22a028e6a9c6763eaddaa3548e5e09d6c95e9 0 1775624641819 1 con
nected
[ec2-user@ip-172-31-70-178 ~]$
```

After recovery — Node 2 rejoined as slave; cluster fully restored to 3 masters + 3 replicas

Explanation:

The baseline confirms excellent Redis Cluster performance on t3.micro: 4,799 ops/sec with 2.04ms average and zero errors. Same-AZ placement eliminates cross-AZ overhead.

During failover three phases occurred: detection (0-5s after kill), election (Node 6 receives votes and is promoted), and recovery (clients reconnect to new master). The most important insight is P99 latency — only 2.4% worse during failover. This means 99% of operations were completely unaffected. The 1,310ms max spike and 318 errors were isolated to the failover window only, demonstrating that a failure in one shard does not degrade operations on the other two shards.

TASK 6: GRADUATE EXTENSION - PERSISTENCE AND RECOVERY

Capability Added:
Both RDB (Redis Database) snapshotting and AOF (Append-Only File) logging were enabled and verified. The recovery test writes keys, forces a snapshot, kills Redis, restarts it, and confirms all keys are recovered — proving Redis functions as a durable key-value store, not just an ephemeral cache.

Commands Run:

Step 1: Verify persistence settings

```
redis-cli config get save
# Output: 1) "save"  2) "3600 1 300 100 60 10000"

redis-cli config get appendonly
# Output: 1) "appendonly"  2) "yes"
```

Step 2: Write test keys

```
redis-cli -c SET persist:test1 "hello_from_adewole"
redis-cli -c SET persist:test2 "redis_persistence_works"
redis-cli -c SET persist:test3 "project2_complete"
```

Step 3: Force RDB snapshot

```
redis-cli -c BGSAVE
# Output: Background saving started
```

Step 4: Verify keys BEFORE restart

```
redis-cli -c GET persist:test1   # "hello_from_adewole"
redis-cli -c GET persist:test2   # "redis_persistence_works"
redis-cli -c GET persist:test3   # "project2_complete"
```

Step 5: Kill Redis (simulate crash)

```
sudo pkill redis-server
redis-cli ping   # Connection refused
```

Step 6: Restart Redis

```
sudo redis-server /etc/redis/redis.conf
```

Step 7: Verify keys survived restart

```
redis-cli -c GET persist:test1   # "hello_from_adewole"    <- RECOVERED
redis-cli -c GET persist:test2   # "redis_persistence_works" <- RECOVERED
redis-cli -c GET persist:test3   # "project2_complete"      <- RECOVERED
```

Step 8: Confirm persistence files on disk

```
ls -lh /var/lib/redis
# drwxr-xr-x  appendonlydir   <- AOF directory
# -rw-r--r--  dump.rdb  268K  <- RDB snapshot
```

```
# -rw-r--r--  nodes.conf      <- cluster topology
```

Screenshots for Task 6:

```
[ec2-user@ip-172-31-70-178 ~]$ redis-cli -c SET persist:test1 "hello_from_adewole"
redis-cli -c SET persist:test2 "redis_persistence_works"
redis-cli -c SET persist:test3 "project2_complete"
redis-cli -c BGSAVE
OK
OK
OK
Background saving started
[ec2-user@ip-172-31-70-178 ~]$ redis-cli -c GET persist:test1
redis-cli -c GET persist:test2
redis-cli -c GET persist:test3
"hello_from_adewole"
"redis_persistence_works"
"project2_complete"
[ec2-user@ip-172-31-70-178 ~]$
```

Keys written and BGSAVE triggered — all 3 values confirmed BEFORE restart

```
[ec2-user@ip-172-31-70-178 ~]$ sudo pkill redis-server
[ec2-user@ip-172-31-70-178 ~]$ redis-cli ping
Could not connect to Redis at 127.0.0.1:6379: Connection refused
[ec2-user@ip-172-31-70-178 ~]$ sudo redis-server /etc/redis/redis.conf
[ec2-user@ip-172-31-70-178 ~]$ redis-cli -c GET persist:test1
redis-cli -c GET persist:test2
redis-cli -c GET persist:test3
"hello_from_adewole"
"redis_persistence_works"
"project2_complete"
[ec2-user@ip-172-31-70-178 ~]$
```

All 3 keys fully recovered AFTER Redis restart — data survived process kill

```
[ec2-user@ip-172-31-70-178 ~]$ redis-cli config get dir
redis-cli config get dbfilename
redis-cli config get appendfilename
1) "dir"
2) "/var/lib/redis"
1) "dbfilename"
2) "dump.rdb"
1) "appendfilename"
2) "appendonly.aof"
[ec2-user@ip-172-31-70-178 ~]$ ls -lh $(redis-cli config get dir | tail -1)
total 272K
drwxr-xr-x. 2 root root  103 Apr  8 05:11 appendonlydir
-rw-r--r--. 1 root root 268K Apr  8 05:11 dump.rdb
-rw-r--r--. 1 root root 1.2K Apr  8 05:10 nodes.conf
[ec2-user@ip-172-31-70-178 ~]$
```

/var/lib/redis showing dump.rdb (268K RDB snapshot) and appendonlydir (AOF) on disk

Explanation:

Running both RDB and AOF together is the recommended production configuration. AOF provides near-zero data loss on crashes (at most ~1 second of writes), while RDB provides compact point-in-time snapshots for fast disaster recovery. All three keys survived a complete Redis process kill and restart — Redis loaded dump.rdb automatically on startup.

Tradeoffs: AOF adds slight write overhead since every operation is logged to disk. RDB's background fork briefly increases memory usage on write-heavy workloads. For t3.micro instances this is negligible.

Limitation: In a full cluster deployment, persistence must be configured on all 6 nodes since each master owns a distinct subset of hash slots and maintains its own dump.rdb and AOF. This demonstration was performed on Node 1 as representative evidence — the same redis.conf was present on all nodes.

SUMMARY

This project successfully deployed, tested, and evaluated a distributed Redis Cluster key-value store on AWS. All core tasks and the graduate extension were completed:

- Cloud Environment Setup: 6 EC2 t3.micro instances in us-east-1f with security group allowing ports 6379, 16379, and 22.
- Redis Cluster Deployment: 3-master + 3-replica cluster initialized via redis-cli --cluster create with all 16,384 hash slots distributed and cluster_state:ok verified.
- Failover Testing: Manual master kill triggered automatic replica promotion in ~5 seconds. Killed node rejoined as replica on restart with no manual reconfiguration.
- Workload Generator: Python cluster-aware generator using RedisCluster client with 10 concurrent threads performing SET/GET/DEL with per-operation latency measurement.

- Performance Evaluation: Baseline achieved 4,799 ops/sec at 2.04ms avg. Failover caused 27% throughput drop and 1,310ms max spike with only 0.3% error rate. P99 degraded 2.4%, confirming per-shard fault isolation.
- Graduate Extension (Persistence & Recovery): RDB + AOF persistence verified. All test keys survived a complete Redis process kill and restart confirming data durability.

Platform Choice Justification: AWS was chosen because it provides real cloud infrastructure matching production Redis Cluster deployments. The 6 t3.micro instances in the same Availability Zone provide low-latency inter-node communication while staying within free-tier boundaries. All Redis concepts demonstrated apply directly to production Redis Cluster deployment.